RTOSCCU

开发指南

版本号: 1.8

发布日期: 2026.02.09

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.07.08	XAA0249	1. 初始版本
1.1	2023.03.28	XAA0249	1. 添加查找 type 和 id 的方法
1.3	2024.03.07	XAA0312	1. 更新适用产品列表
1.4	2024.10.22	AWA1427	1. 基于新模板更新文档
1.5	2025.05.22	XAA0338	1. 更新适用产品列表、新时钟相关描述及内容格式
1.6	2025.07.26	AWA1982	1. 增加 F135、F136 支持
1.7	2025.11.19	XAA0338	1. 更新文档标题格式 2. 更新文档框架 3. 更新调试方法章节 4. 增加 FAQ 章节
1.8	2026.02.09	XAA0338	1. 更新适用产品列表

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

目录

第 1 章 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 标志说明 1
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 2
1.5 相关术语介绍 2
1.5.1 硬件术语 2
1.5.2 软件术语 2

第 2 章 模块介绍 3
2.1 模块功能 3
2.2 模块配置 3
2.3 源码结构 4
2.3.1 旧驱动框架 4
2.3.2 新驱动框架 5
2.4 软件框架 6

第 3 章 模块接口说明 7
3.1 数据结构 7
3.1.1 hal_clk_type_t 7
3.1.2 hal_clk_status_t 7
3.2 查找某个时钟的 type 和 id 8
3.2.1 查阅芯片手册 8
3.2.2 直接查找源码 8
3.3 hal_clock_get 9
3.4 hal_clock_put 9
3.5 hal_clock_init 10
3.6 hal_clock_is_enabled 10
3.7 hal_clock_enable 10
3.8 hal_clock_disable 11
3.9 hal_clk_recalc_rate 11
3.10 hal_clk_round_rate 11
3.11 hal_clk_get_rate 12
3.12 hal_clk_set_rate 12
3.13 hal_clk_set_parent 12
3.14 hal_clk_get_parent 13
3.15 hal_clock_get_hosc_freq 13
3.16 hal_clk_ccu_aon_set_freq_trim 13
3.17 hal_clk_ccu_aon_get_freq_trim 14

第 4 章 功能开发 15
4.1 功能概述 15
4.2 开发流程 15
4.2.1 开关某个时钟 15
4.2.2 获取某个时钟的频率 15
4.2.3 设置某个时钟的频率 15
4.2.4 获取某个时钟的时钟源 15
4.2.5 设置某个时钟的时钟源 16
4.3 模块使用示例 16
4.3.1 使能时钟 16
4.3.2 设置时钟频率 16

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

4.3.3 设置父时钟 . . . 17

第 5 章 问题调试 18
5.1 调试信息 . . . 18
5.1.1 寄存器直接读取 . . . 18
5.1.2 使用 hal_ccmu/hal_ccu 命令 . . . 18
5.1.3 问题定位流程 . . . 19
5.2 日志分析 . . . 19
5.2.1 典型日志示例 . . . 19

第 6 章 FAQ 20
6.1 常见问题 . . . 20
6.1.1 时钟使能失败 . . . 20
6.1.2 设置时钟频率失败 . . . 20
6.1.3 确认时钟是否已正确使能 . . . 21
6.2 一号通 FAQ 列表 . . . 22

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

表格

表 1-1 适用产品列表 1
表 1-2 地址与数据描述方法约定 2
表 1-3 数值单位约定 2
表 1-4 硬件术语 2
表 1-5 软件术语 2
表 3-1 模块接口列表 7
表 3-2 时钟域与文件对应关系 8

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 1 章 前言

1.1 文档简介

本文档介绍 RTOS 平台 CCU 驱动的接口及使用方法。

1.2 目标读者

- RTOS CCU 驱动开发工程师
- RTOS CCU 驱动使用工程师

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V459	Melis	hal_clk.c
F133	Melis	hal_clk.c
R328	FreeRTOS	hal_clk.c
R329-DSP	FreeRTOS	hal_clk.c
R128	FreeRTOS	hal_clk.c
XR875	FreeRTOS	hal_clk.c
MR527	FreeRTOS	hal_clk.c
V821	FreeRTOS	hal_clk.c
MR153	FreeRTOS	hal_n_clk.c
T153	FreeRTOS	hal_n_clk.c
F135	Melis	hal_n_clk.c
F136	Melis	hal_n_clk.c
V861	FreeRTOS	hal_n_clk.c
V881	FreeRTOS	hal_n_clk.c
V838	FreeRTOS	hal_n_clk.c
F102	Melis	hal_n_clk.c
F101	FreeRTOS	hal_n_clk.c

1.4 文档约定

1.4.1 标志说明

注意

提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示（寄存器描述除外）。
X	00X，XX1	数据描述中，X 代表 0 或 1（例如，00X 代表 000 或 001；XX1 代表 001、011、101 或 111）。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K 1 M 1 G	1024 1 048 576 1 073 741 824
频率，数据速率等	1 K 1 M 1 G	1000 1 000 000 1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
晶振	晶体振荡器，晶振具有固定振荡频率，如 32K/24MHz 等，是芯片所有时钟的源头
PLL	锁相环，利用输入信号和反馈信号的差异提升频率输出
CCU	Clock Control Unit（时钟控制单元）的简称，用于管理和分配时钟信号

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
CLK	Clock（时钟）的简称，表示时钟信号或时钟源
HAL	Hardware Abstraction Layer，硬件抽象层

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 2 章 模块介绍

2.1 模块功能

CCU（Clock Control Unit，时钟控制单元）模块负责管理和控制 SOC 系统的时钟信号，确保各个模块在正确的频率下工作，从而优化性能、功耗和稳定性。

主要作用和功能：

1. 时钟生成

主时钟源：CCU 通常会生成系统的主时钟信号，这个信号是整个系统的基础时钟。主时钟源可以来自外部晶振、PLL（Phase-Locked Loop，锁相环）或其他时钟源。

内部时钟源：CCU 还可以生成内部时钟信号，这些信号可能用于特定的子系统或模块。

2. 时钟分频

分频器：CCU 可以通过分频器将主时钟信号分频，生成不同频率的时钟信号。这使得不同的模块可以根据需要运行在不同的频率下，从而优化性能和功耗。

动态调整：在某些情况下，CCU 可以动态调整分频比，以适应系统负载的变化，进一步优化功耗和性能。

3. 时钟门控

时钟门控：CCU 可以通过时钟门控技术关闭某些模块的时钟信号，从而在不需要时节省功耗。这对于低功耗设计尤为重要。

按需启用：当模块需要工作时，CCU 可以重新启用时钟信号，确保模块正常运行。

4. 时钟配置和管理

通过寄存器来配置和管理时钟信号。

2.2 模块配置

menuconfig 配置说明

1. 普通 SDK 环境

```
Drivers Options  --->
    soc related device drivers  --->
        [*] enable ccmu driver
        [*]   enable sunxi-ng ccmu driver
        [ ] enable sunxi-ng ccmu hal APIs test command
        [ ] donot disable parent when clk disable
```

说明

sun8iw18p1/sun8iw19p1/sun8iw21p1 平台不需要配置 “enable sunxi-ng ccmu driver”，其它平台默认配置。

2. R128/XR875 SDK

配置条目大同小异

```
Drivers Options  --->
    soc related device drivers  --->
        [*] enable ccmu driver
        [ ]   enable sunxi ccmu driver
        [*]   enable sunxi-ng ccmu driver
        [ ] enable sunxi-ng ccmu hal APIs test command
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    [ ] donot disable parent when clk disable
```

2.3 源码结构

我司现支持两套时钟框架：

旧时钟框架：适用于 MR527、T536、T113 等历史平台

新时钟框架：适用于 MR153、F135 平台

另外 V821、MR536 和 R128 平台既可使用新时钟框架，也可使用旧时钟框架

 说明

新驱动框架：优化时钟定义的内存占用，优化 API 执行耗时，提升易用性。

2.3.1 旧驱动框架

1. 驱动源码

```
hal/source/ccmu/
├── common_ccmu.h
├── hal_clk.c
├── hal_reset.c
├── Kconfig
├── Makefile
├── platform_ccmu.h
├── platform_rst.h
├── sunxi     #旧版本驱动，sun8iw18p1/sun8iw19p1/sun8iw21p1 平台使用
│   ├── clk.c
│   ├── clk_factors.c
│   ├── clk_factors.h
│   ├── clk.h
│   ├── clk_periph.c
│   ├── clk_periph.h
│   ├── Makefile
│   ├── platform_clk.h
│   ├── sun8iw18p1
│   │   ├── clk_sun8iw18.c
│   │   ├── clk_sun8iw18.h
│   │   └── Makefile
│   ├── sun8iw19p1
│   │   ├── clk_sun8iw19.c
│   │   ├── clk_sun8iw19.h
│   │   └── Makefile
│   └── sun8iw21p1
│       ├── clk_sun8iw21.c
│       ├── clk_sun8iw21.h
│       └── Makefile
└── sunxi-ng   #新版本驱动，新平台一般使用该驱动
    ├── ccu.c
    ├── ccu_common.c
    ├── ccu_common.h
    ├── ccu_div.c
    ├── ccu_div.h
    ├── ccu_frac.c
    ├── ccu_frac.h
    ├── ccu_gate.c
    ├── ccu_gate.h
    ├── ccu.h
    ├── ccu_mp.c
    ....
    ├── ccu_slab.h
    ├── ccu-sun20iw2-aon.c
    ├── ccu-sun20iw2-aon.h
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    ├── ccu-sun20iw2.c
    ├── ccu-sun20iw2.h
    ├── ccu-sun20iw2-r.c
    ├── ccu-sun20iw2-r.h
    ├── ccu-sun300iw1-aon.c
    ├── ccu-sun300iw1-aon.h
    ├── ccu-sun300iw1-app.c
    ├── ccu-sun300iw1-app.h
    ├── ccu-sun300iw1.c
    ├── ccu-sun55iw3-rtc.h
    ├── ccu-sun55iw6.c
    ├── ccu-sun55iw6.h
    ├── ccu-sun55iw6-r.c
    ├── ccu-sun55iw6-r.h
    ├── ccu-sun55iw6-rtc.c
    ├── ccu-sun55iw6-rtc.h
....
```

sunxi-ng 目录中，因为各平台设计差异，会有不同的平台源文件，例如：

- sun20iw2 有 ccu-sun20iw2-aon.c/ccu-sun20iw2.c/ccu-sun20iw2-r.c
- sun55iw6 有 ccu-sun55iw6.c/ccu-sun55iw6-r.c/ccu-sun55iw6-rtc.c
- sun300iw1 有 ccu-sun300iw1-aon.c/ccu-sun300iw1-app.c/ccu-sun300iw1.c

2. 驱动 APIs 声明头文件

```
include/hal/
└── hal_clk.h
```

3. 驱动 APIs 测试代码

```
hal/test/ccmu/ ---- 旧驱动APIs测试代码
├── Makefile
├── test_ccmu.c
└── test_conf.h

hal/test/ccmu-ng/ ---- sunxi-ng驱动APIs测试代码
├── Makefile
└── test_ng_ccmu.c
```

2.3.2 新驱动框架

1. 驱动源码

```
hal/source/ccu/
├── clk_core
│   ├── aw_ccu.c
│   ├── aw_ccu.h
│   ├── ccu_div.c
│   ├── ccu_div.h
│   ├── ccu_gate.c
│   ├── ccu_gate.h
│   ├── ccu_multiplier.c
│   ├── ccu_multiplier.h
│   ├── ccu_mux.c
│   ├── ccu_mux.h
│   ├── ccu_pll.c
│   ├── ccu_pll.h
│   ├── ccu_reset.c
│   ├── ccu_reset.h
│   ├── clk_common.h
│   ├── clk_core.c
│   ├── clk_core.h
│   ├── clk_fixed_factor.c
│   ├── clk_source.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
|   ├── clk_source.h
|   ├── Makefile
|   ├── reset_common.h
├── hal_n_clk.c
├── hal_reset.c
├── Kconfig
├── Makefile
├── platform
|   ├── Makefile
|   ├── platform_ccu.h
|   ├── platform_common.c
|   ├── platform_common.h
|   ├── platform_info.h
|   ├── sun300iw1
|   |   ├── Makefile
|   |   ├── sun300iw1_aon_ccu.c
|   |   ├── sun300iw1_aon_ccu.h
|   |   ├── sun300iw1_app_ccu.c
|   |   ├── sun300iw1_app_ccu.h
|   |   ├── sun300iw1_ccu_id.h
|   |   ├── sun300iw1_src_ccu.c
|   |   └── sun300iw1_src_ccu.h
|   ├── sun55iw6
|   |   ├── ....
|   ├── sun8iw22
|   |   ├── Makefile
|   |   ├── sun8iw22_ccu_id.h
|   |   ├── sun8iw22_prcm_ccu.c
|   |   ├── sun8iw22_prcm_ccu.h
|   |   ├── sun8iw22_src_ccu.c
|   |   ├── sun8iw22_src_ccu.h
|   |   ├── sun8iw22_sys_ccu.c
|   |   ├── sun8iw22_sys_ccu.h
....
```

2. 驱动 APIs 声明头文件

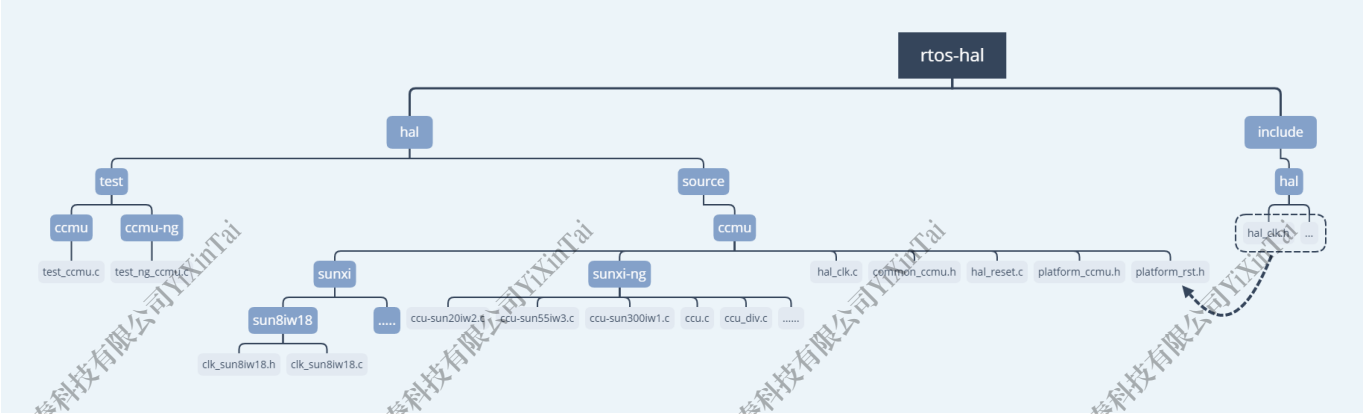
```
include/hal/
└── hal_clk.h
```

3. 驱动 APIs 测试代码

```
hal/test/ccu-ng/
├── Makefile
└── test_ccu.c
```

2.4 软件框架

图 2-1: 图 1：RTOS CCU 驱动框架



版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 3 章 模块接口说明

表 3-1: 模块接口列表

API	解释说明
hal_clock_init	初始化 CCU 驱动
hal_clock_is_enabled	判断指定时钟是否已经打开
hal_clock_enable	打开指定时钟
hal_clock_disable	关闭指定时钟
hal_clk_recalc_rate	重新计算指定时钟的频率
hal_clk_round_rate	设置一个跟指定频率最接近的时钟频率
hal_clk_get_rate	获取指定时钟频率（可能不是实时的）
hal_clk_set_rate	设置指定时钟的频率
hal_clk_set_parent	设置指定时钟的父时钟
hal_clk_get_parent	获取指定时钟的父时钟
hal_clock_get	获取指定时钟句柄
hal_clock_put	释放指定时钟句柄
hal_clock_get_hosc_freq	获取 hosc 时钟频率

3.1 数据结构

3.1.1 hal_clk_type_t

描述时钟类型，用于获取时钟。

```
typedef enum
{
    HAL_SUNXI_FIXED_CCU = 0,
    HAL_SUNXI_RTC_CCU,
    HAL_SUNXI_CCU,
    HAL_SUNXI_AON_CCU,
    HAL_SUNXI_R_CCU,
    HAL_SUNXI_DSP,
    HAL_SUNXI_CCU_NUMBER,
} hal_clk_type_t;
```

3.1.2 hal_clk_status_t

描述函数运行状态。

```
typedef enum
{

    HAL_CLK_STATUS_DISABLED = -1,
    HAL_CLK_STATUS_ENABLED = 0,
    HAL_CLK_STATUS_ERROR_CLK_FACTOR_REFUSED = -11,
    HAL_CLK_STATUS_ERROR_CLK_NEED_DISABLED  = -10,
    HAL_CLK_STATUS_ERROR_CLK_PARENT_DISABLED  = -9,
    HAL_CLK_STATUS_ERROR_CLK_ENABLED_FAILED  = -8,
    HAL_CLK_STATUS_ERROR_CLK_ROUND_FAILED = -7,
    HAL_CLK_STATUS_ERROR_CLK_SET_RATE_REFUSED = -6,
    HAL_CLK_STATUS_ERROR_CLK_NOT_FOUND  = -5,
    HAL_CLK_STATUS_ERROT_CLK_UNDEFINED  = -4,
    HAL_CLK_STATUS_UNINITIALIZED = -3,      /**< Uninitialized clock driver. */
    HAL_CLK_STATUS_INVALID_PARAMETER = -2,    /**< Invalid parameter. */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    HAL_CLK_STATUS_ERROR = -1,              /**< Unknown error. */
    HAL_CLK_STATUS_OK = 0,                  /**< Successful. */
} hal_clk_status_t;
```

3.2 查找某个时钟的 type 和 id

技巧

方法 1 和方法 2 不是完全独立的，建议用户先直接用方法 2，如果未查出来，再结合方法 1。

3.2.1 查阅芯片手册

查询对应型号的 User Manual 时钟相关章节，确认您要操作的时钟在哪个域，再找到对应的代码文件。时钟域与文件的对应关系参考如下：

表 3-2: 时钟域与文件对应关系

时钟所在域	时钟文件名	type
CCMU	ccu-sunXXiwX.c	HAL_SUNXI_CCU
CCMU_APP	ccu-sunXXiwX-app.c	HAL_SUNXI_CCU
CCMU_AON	ccu-sunXXiwX-aon.c	HAL_SUNXI_AON_CCU
PRCM	ccu-sunXXiwX-r.c	HAL_SUNXI_R_CCU
DSP_PRCM	sunXXiwX-dsp.c	HAL_SUNXI_DSP
SYSRTC	sunXXiwX-rtc.c	HAL_SUNXI_RTC_CCU

3.2.2 直接查找源码

直接到对应平台时钟文件下搜索，以 R128/XR875 平台为例，查找 UART2。

步骤 1 R128/XR875 平台对应 sun20iw2，到 sunxi-ng 目录下找到 ccu_sun20iw2_xx.h 几个头文件，在里面查找 UART2 字样。

图 3-1: 图 2：ccu_sun20iw2 头文件

```
1: ccu-sun20iw2.h
 1 #ifndef _CCU_SUN20IW2_H_
 2 #define _CCU_SUN20IW2_H_
 3
 4 #define SUNXI20_CCU_BASE      0x4003C000
 5
 6 enum {
 7     CLK_OSC12M = 0,
 8     CLK_BUS_PWM,
 9     CLK_BUS_TWI1,
10     CLK_BUS_TWI0,
11     CLK_BUS_UART0,
12     CLK_BUS_UART1,
13     CLK_BUS_UART2,
14     CLK_BUS_SDC0,
```

步骤 2 在 ccu_sun20iw2.h 中看到 CLK_BUS_UART2，这个对应 clk id，再查看对应的 ccu_sun20iw2.c，搜索.clk_type，如下图所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

图 3-2: 图 3：ccu_sun20iw2 源码

```
539 static const struct sunxi_ccu_desc sun20iw2_ccu_desc =
540 {
541     .ccu_clks    = sun20iw2_ccu_clks,
542     .num_ccu_clks    = ARRAY_SIZE(sun20iw2_ccu_clks),
543
544     .hw_clks     = &sun20iw2_hw_clks,
545     .clk_type    = HAL_SUNXI_CCU,
546
547     .resets      = sun20iw2_ccu_resets,
548     .reset_type = HAL_SUNXI_RESET,
549     .num_resets = ARRAY_SIZE(sun20iw2_ccu_resets),
550 };
```

可以看出 clk type 是 HAL_SUNXI_CCU。

步骤 3 因此 UART2 时钟的 clk type 是 HAL_SUNXI_CCU，clk id 是 CLK_BUS_UART2。

3.3 hal_clock_get

函数原型:

```
hal_clk_t hal_clock_get(hal_clk_type_t type, hal_clk_id_t id)
```

作用:

获取指定时钟句柄。

参数:

type：时钟类型，具体查看 hal_clk_type_t 结构体。

id：时钟 ID，具体取值根据平台不同而不同，请参考 “3.2 章节” 。

返回:

hal_clk_t：获取到的时钟句柄（非 NULL 表示成功，NULL 表示失败）。

3.4 hal_clock_put

函数原型:

```
hal_clk_status_t hal_clock_put(hal_clk_t clk)
```

作用:

释放指定时钟句柄。

参数:

clk：要操作的时钟句柄。

返回:

hal_clk_status_t：函数运行状态。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

3.5 hal_clock_init

函数原型:

```
void hal_clock_init(void)
```

作用:

初始化 CCU 驱动。

参数:

void

返回:

void

3.6 hal_clock_is_enabled

函数原型:

```
hal_clk_status_t hal_clock_is_enabled(hal_clk_t clk)
```

作用:

判断指定时钟是否已经打开。

参数:

clk：时钟句柄。

返回:

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_STATUS_ENABLED：时钟已打开。

HAL_CLK_STATUS_DISABLED：时钟已关闭。

3.7 hal_clock_enable

函数原型:

```
hal_clk_status_t hal_clock_enable(hal_clk_t clk)
```

作用:

打开指定时钟。

参数:

clk：时钟句柄。

返回:

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_STATUS_ERROR_CLK_UNDEFINED：clock id 未定义。

HAL_CLK_STATUS_ENABLED：成功。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

3.8 hal_clock_disable

函数原型：

```
hal_clk_status_t hal_clock_disable(hal_clk_t clk)
```

作用：

关闭指定时钟。

参数：

clk：时钟句柄。

返回：

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_STATUS_ERROR_CLK_UNDEFINED：clock id 未定义。

HAL_CLK_STATUS_DISABLED：成功。

3.9 hal_clk_recalc_rate

函数原型：

```
u32 hal_clk_round_rate(hal_clk_t clk, u32 rate)
```

作用：

重新计算指定时钟的频率。

参数：

clk：时钟句柄。

返回：

recalc_rate：计算过后的时钟频率。

3.10 hal_clk_round_rate

函数原型：

```
u32 hal_clk_round_rate(hal_clk_t clk, u32 rate)
```

作用：

调整时钟频率。

参数：

clk：时钟句柄。

rate：指定的频率。

返回：

返回跟指定频率最接近的频率。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

3.11 hal_clk_get_rate

函数原型：

```
u32 hal_clk_get_rate(hal_clk_t clk)
```

作用：

获取指定时钟频率（可能不是实时的）。

参数：

clk：时钟句柄。

返回：

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_RATE_UNINITIALIZED：设置失败。

3.12 hal_clk_set_rate

函数原型：

```
hal_clk_status_t hal_clk_set_rate(hal_clk_t clk, u32 rate)
```

作用：

设置指定时钟的频率。

参数：

clk：时钟句柄。

rate：设置的频率。

返回：

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_STATUS_ERROR_CLK_SET_RATE_REFUSED：拒绝设置。

HAL_CLK_STATUS_ERROR_CLK_UNDEFINED：clock id 未定义。

HAL_CLK_STATUS_OK：成功。

3.13 hal_clk_set_parent

函数原型：

```
hal_clk_status_t hal_clk_set_parent(hal_clk_t clk, hal_clk_t parent)
```

作用：

设置指定时钟的父时钟。

参数：

clk：时钟句柄。

parent：父时钟的时钟句柄。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

返回：

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_STATUS_ERROR_CLK_SET_RATE_REFUSED：拒绝设置。

HAL_CLK_STATUS_ERROR_CLK_UNDEFINED：clock id 未定义。

HAL_CLK_STATUS_OK：成功。

3.14 hal_clk_get_parent

函数原型：

```
hal_clk_t hal_clk_get_parent(hal_clk_t clk)
```

作用：

获取指定时钟的父时钟。

参数：

clk：时钟句柄。

返回：

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_UNINITIALIZED：时钟未初始化。

parent clk：父时钟时钟句柄。

3.15 hal_clock_get_hosc_freq

函数原型：

```
uint32_t hal_clock_get_hosc_freq(void)
```

作用：

动态获取 hosc 的频率。

参数：

void

返回：

hosc 时钟频率。

3.16 hal_clk_ccu_aon_set_freq_trim

函数原型：

```
hal_clk_status_t hal_clk_ccu_aon_set_freq_trim(uint32_t value)
```

作用：

配置 ccu_aon dcxo 频偏值，无线模块使用。

参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

value：设置的 dcxo 频偏。

返回：

HAL_CLK_STATUS_INVALID_PARAMETER：参数非法。

HAL_CLK_RATE_UNINITIALIZED：设置失败。

HAL_CLK_STATUS_OK：成功。

3.17 hal_clk_ccu_aon_get_freq_trim

函数原型：

```
uint32_t hal_clk_ccu_aon_get_freq_trim(void)
```

作用：

获取 ccu_aon dcxo 频偏值，无线模块使用。

参数：

void

返回：

dcxo 频偏。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 4 章 功能开发

4.1 功能概述

CCU 驱动提供了以下核心功能：

- 开关时钟；
- 设置时钟源；
- 获取时钟频率；
- 获取时钟源；
- 设置时钟频率；

4.2 开发流程

4.2.1 开关某个时钟

步骤 1 确定时钟的 clk type 和 clk id。

步骤 2 调用 hal_clock_get(clk_type, clk_id)，根据 clk_type 和 clk_id 获取时钟句柄。

步骤 3 调用 hal_clock_enable 打开时钟。

步骤 4 调用 hal_clock_disable 关闭时钟。

步骤 5 调用 hal_clock_put 释放时钟句柄。

4.2.2 获取某个时钟的频率

步骤 1 确定时钟的 clk type 和 clk id。

步骤 2 调用 hal_clock_get(clk_type, clk_id)，根据 clk_type 和 clk_id 获取时钟句柄。

步骤 3 调用 hal_clk_get_rate，根据时钟句柄获取到时钟频率。

步骤 4 调用 hal_clock_put 释放时钟句柄。

4.2.3 设置某个时钟的频率

步骤 1 确定时钟的 clk type 和 clk id。

步骤 2 调用 hal_clock_get(clk_type, clk_id)，根据 clk_type 和 clk_id 获取时钟句柄。

步骤 3 调用 hal_clk_set_rate，根据时钟句柄设置时钟频率。

步骤 4 调用 hal_clock_put 释放时钟句柄。

4.2.4 获取某个时钟的时钟源

步骤 1 确定时钟的 clk type 和 clk id。

步骤 2 调用 hal_clock_get(clk_type, clk_id)，根据 clk_type 和 clk_id 获取时钟句柄。

步骤 3 调用 hal_clk_get_parent，获得时钟源的时钟句柄。

步骤 4 调用 hal_clock_put 释放时钟句柄。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

4.2.5 设置某个时钟的时钟源

步骤 1 确定时钟的 clk type 和 clk id。

步骤 2 调用 hal_clock_get(clk_type, clk_id)，根据 clk_type 和 clk_id 获取时钟句柄。

步骤 3 调用 hal_clk_set_parent 重新设置时钟源。

步骤 4 调用 hal_clock_put 释放时钟句柄。

4.3 模块使用示例

4.3.1 使能时钟

参考 hal_uart.c 使用，截取部分关键代码：

```
static int uart_clk_init(int bus, bool enable)
{
    ...
    hal_clk_type_t clk_type = HAL_SUNXI_CCU;    /* 定义时钟类型 */
    hal_clk_id_t clk_id;                        /* 定义时钟ID */
    hal_clk_t clk;                              /* 定义时钟句柄 */
    hal_clk_status_t ret;
    ...
    clk_id = uart_msg[bus].clk_id;              /* 获取UART对应的时钟ID */

    /* 根据clk_type和clk_id获取时钟句柄 */
    clk = hal_clock_get(clk_type, clk_id);
    if (!clk) {
        UART_ERR("uart%d fail to get bus clk\n", bus);
        return -1;
    }

    /* 使能时钟 */
    ret = hal_clock_enable(clk);
    if (ret) {
        UART_ERR("[uart%d] couldn't enable clk!\n", bus);
        return -1;
    }
    ...
}
```

技巧

当 hal_clock_enable() 返回失败时，您需要检查：1. 父时钟是否已使能 2. 时钟 ID 是否正确 3. 时钟源配置是否正常

4.3.2 设置时钟频率

参考 cpufreq.c，截取部分关键代码。

路径：lichee/rtos/arch/risc-v/sun20iw2p1/cpufreq.c

```
static int set_first_div_freq(uint32_t target_freq, hal_clk_t next_clk)
{
    int ret = 0;
    hal_clk_t clk_dpll1_rv_div;

    /* 获取时钟句柄 */
    clk_dpll1_rv_div = hal_clock_get(HAL_SUNXI_AON_CCU, CLK_CK1_C906);
    if (!clk_dpll1_rv_div)
    {
        ret = -1;  /* 获取时钟句柄失败 */
        goto err_get_clk_dpll3_div;
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    }
    /* 设置时钟频率 */
    ret = hal_clk_set_rate(clk_dpll1_rv_div, target_freq);
    if (HAL_CLK_STATUS_OK != ret)
    {
        ret = -2;
        goto err_set_dpll3_div_rate;
    }
    ....
}
```

技巧

设置时钟频率时的注意事项：

1. 目标频率必须在硬件支持的范围内
2. 某些时钟在使能状态下不允许修改频率，需要先调用 hal_clock_disable() 关闭
3. 设置完成后建议通过 hal_clk_get_rate() 验证实际频率

4.3.3 设置父时钟

参考 hal_uart.c 使用，截取部分关键代码：

```
static int uart_clk_init(int bus, bool enable)
{
    ...
    hal_clk_t pclk;              /* 父时钟句柄 */
    hal_clk_status_t ret;
    ...
    /* 获取当前时钟句柄 */
    clk = hal_clock_get(HAL_SUNXI_CCU, UART_SCLK);
    if (!clk) {
        UART_ERR("uart%d fail to get SCLK\n", bus);
        return -1;
    }

    /* 获取父时钟时钟句柄 */
    pclk = hal_clock_get(HAL_SUNXI_AON_CCU, UART_PCLK);
    if (!pclk) {
        UART_ERR("uart%d fail to get PCLK\n", bus);
        return -1;
    }

    /* 设置UART_SCLK的父时钟为UART_PCLK */
    ret = hal_clk_set_parent(clk, pclk);
    if (ret != HAL_CLK_STATUS_OK) {
        UART_ERR("uart%d fail to setparent\n", bus);
        return -1;
    }
    ...
}
```

技巧

切换父时钟时的建议：1. 建议在时钟禁用状态下切换父时钟 2. 切换后需要重新配置时钟频率 3. 验证新的父时钟已正确使能

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 5 章 问题调试

5.1 调试信息

CCU 驱动主要是管理和配置时钟，调试主要是检查时钟是否能够正确设置和获取时钟信息。

5.1.1 寄存器直接读取

查阅芯片手册，查找相应时钟寄存器，通过 p 命令读取寄存器确认时钟是否符合预期。

示例：检查 UART2 时钟是否已使能

```
# 假设CCU基地址为0x02001000，UART2_CLK寄存器偏移为0x90C
p 0x0200190C
```

检查输出结果：- Bit[31]：CLK_GATING，1 表示时钟开门，0 表示时钟关闭 - Bit[25:24]：CLK_SRC_SEL，时钟源选择 - Bit[4:0]：DIV_M，分频系数

5.1.2 使用 hal_ccmu/hal_ccu 命令

步骤 1 配置 menuconfig

旧时钟框架配置 CONFIG_HAL_TEST_CCU：

```
Drivers Options  --->
    soc related device drivers  --->
        CCMU Devices  --->
            [*] enable sunxi-ng ccmu hal APIs test command
```

新时钟框架配置 HAL_TEST_CCU_NG：

```
Drivers Options  --->
    soc related device drivers  --->
        CCU Driver(Next Generation)  --->
            [*]   enable CCU hal APIs test command
```

步骤 2 通过 hal_ccmu/hal_ccu 命令查看时钟树信息

图 5-1: 图 4：时钟树信息

```
hspsram-ctrl                HAL_SUNXI_CCU       ckpll-hspsram-sel   200000000
lspsram-gate                HAL_SUNXI_CCU       hosc                40000000
lspsram-sel                 HAL_SUNXI_CCU       hosc                40000000
lspsram-div                 HAL_SUNXI_CCU       lspsram-sel         40000000
lspsram-ctrl                HAL_SUNXI_CCU       lspsram-div         20000000
ir_rx                       HAL_SUNXI_CCU       hosc                40000000
ir_tx                       HAL_SUNXI_CCU       hosc                40000000
bus-ohci0                   HAL_SUNXI_CCU       osc12M              20000000
bus-csi-jpe                 HAL_SUNXI_CCU       osc12M              20000000
bus-ohci0                   HAL_SUNXI_CCU       osc12M              20000000
bus-otg                     HAL_SUNXI_CCU       hosc                40000000
bus-smcard                  HAL_SUNXI_CCU       hosc                40000000
bus-hspsram-ctrl            HAL_SUNXI_CCU       hosc                40000000
bus-ledc                    HAL_SUNXI_CCU       hosc                40000000
g2d                         HAL_SUNXI_CCU       hosc                40000000
de                          HAL_SUNXI_CCU       hosc                40000000
lcd                         HAL_SUNXI_CCU       hosc                40000000
hosc                        HAL_SUNXI_AON_CCU   dcxo40M             40000000
hosc-detect                 HAL_SUNXI_AON_CCU   hosc                40000000
dpll1                       HAL_SUNXI_AON_CCU   hosc                1920000000
dpll2                       HAL_SUNXI_AON_CCU   hosc                1920000000
dpll3                       HAL_SUNXI_AON_CCU   hosc                1600000000
rfip0-dpll                  HAL_SUNXI_AON_CCU   dpll1               160000000
rfip1-dpll                  HAL_SUNXI_AON_CCU   dpll3               133333333
pll-audio                   HAL_SUNXI_AON_CCU   hosc                98333333
pll-audio1x                 HAL_SUNXI_AON_CCU   pll-audio           24583333
pll-audio2x                 HAL_SUNXI_AON_CCU   pll-audio           49166666
ck1-usb                     HAL_SUNXI_AON_CCU   hosc                40000000
ck1-aud                     HAL_SUNXI_AON_CCU   dpll1               22588236
ck1-dev                     HAL_SUNXI_AON_CCU   dpll1               384000000
ck1-lspsram                 HAL_SUNXI_AON_CCU   dpll1               240000000
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

步骤 3 分析时钟树信息

时钟树输出包含以下关键信息：- 时钟名称 - 时钟频率 - 使能状态 - 父时钟关系

5.1.3 问题定位流程

当 UART 通信异常时，您可以按照以下流程排查：

```
1. 使用hal_ccu tree查看UART时钟状态

2. 检查时钟是否已使能？
   └─ 否    调用hal_clock_enable()
   └─ 是    继续下一步

3. 检查时钟频率是否匹配波特率？
   └─ 不匹配    调用hal_clk_set_rate()
   └─ 匹配    继续下一步

4. 检查父时钟是否正常？
   └─ 异常    检查父时钟配置
   └─ 正常    检查其他硬件配置
```

5.2 日志分析

5.2.1 典型日志示例

示例 1：时钟使能失败

```
[CCU][ERROR] hal_clock_enable: parent clock disabled
[UART][ERROR] uart0 couldn't enable clk!
```

原因分析：父时钟未使能

解决方法：

```
/* 先使能父时钟 */
hal_clock_enable(parent_clk);
/* 再使能子时钟 */
hal_clock_enable(clk);
```

示例 2：设置频率失败

```
[CCU][ERROR] hal_clk_set_rate: clock is enabled, cannot change rate
[CCU][ERROR] requested rate 100000000, but hw doesn't support
```

原因分析：或者硬件不支持该频率

解决方法：根据父时钟以及整条时钟链路的分频系数计算目标频率是否支持

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 6 章 FAQ

6.1 常见问题

6.1.1 时钟使能失败

问题现象：

调用了 hal_clock_enable() 函数，返回成功，但是时钟仍然没有输出，模块无法正常工作。

可能的原因：

1. 父时钟未使能或父时钟频率为 0
2. 时钟树节点配置不正确（如分频系数设置错误）
3. 时钟源选择错误
4. 硬件管脚复用功能配置错误

处理措施：

1. **检查父时钟状态**

```
/* 获取父时钟 */
hal_clk_t parent = hal_clk_get_parent(clk);
if (parent) {
    /* 检查父时钟是否已使能 */
    hal_clk_status_t status = hal_clock_is_enabled(parent);
    if (status != HAL_CLK_STATUS_ENABLED) {
        printf("Parent clock is disabled!\n");
        hal_clock_enable(parent);
    }
    /* 检查父时钟频率 */
    uint32_t parent_rate = hal_clk_get_rate(parent);
    printf("Parent clock rate: %u Hz\n", parent_rate);
}
```

3. **检查寄存器配置**

根据芯片手册查找对应的时钟寄存器，使用 p 命令读取：

```
p 0x02001XXX  # 替换为实际寄存器地址
```

重点检查：- GATING 位是否为 1（时钟开门）- CLK_SRC_SEL 是否正确 - 分频系数是否合理

6.1.2 设置时钟频率失败

问题现象：

尝试设置时钟频率时，函数返回错误。

可能的原因：

1. 时钟在使能状态下，且该时钟不允许在运行时修改频率
2. 目标频率超出硬件支持范围
3. 时钟是固定频率时钟，不允许修改
4. 父时钟频率不支持生成目标频率

处理措施：

1. **在禁用状态下修改频率**

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
/* 1. 先禁用时钟 */
hal_clk_status_t ret = hal_clock_disable(clk);
if (ret != HAL_CLK_STATUS_DISABLED) {
    printf("Failed to disable clock\n");
    return -1;
}

/* 2. 设置频率 */
ret = hal_clk_set_rate(clk, target_rate);
if (ret != HAL_CLK_STATUS_OK) {
    printf("Failed to set rate: %d\n", ret);
    return -1;
}

/* 3. 验证实际频率 */
uint32_t actual_rate = hal_clk_get_rate(clk);
printf("Requested: %u Hz, Actual: %u Hz\n", target_rate, actual_rate);

/* 4. 重新使能时钟 */
ret = hal_clock_enable(clk);
if (ret != HAL_CLK_STATUS_ENABLED) {
    printf("Failed to enable clock\n");
    return -1;
}
```

2. **使用 hal_clk_round_rate() 获取最接近的支持频率**

```
/* 获取最接近的支持频率 */
uint32_t round_rate = hal_clk_round_rate(clk, target_rate);
printf("Requested: %u Hz, Rounded: %u Hz\n", target_rate, round_rate);

/* 使用round_rate设置 */
ret = hal_clk_set_rate(clk, round_rate);
```

3. **检查时钟频率范围**

查阅用户手册确认：- 该时钟的最大/最小频率限制 - 支持的频率列表 - 分频系数范围

6.1.3 确认时钟是否已正确使能

问题场景：

在调试过程中需要确认 UART、SPI 等模块的时钟是否已正确使能。

处理方法：

1. **使用 API 检查**

```
hal_clk_t clk = hal_clock_get(HAL_SUNXI_CCU, CLK_BUS_UART0);
if (!clk) {
    printf("Failed to get clock\n");
    return -1;
}

hal_clk_status_t status = hal_clock_is_enabled(clk);
if (status == HAL_CLK_STATUS_ENABLED) {
    printf("Clock is enabled\n");
    /* 获取当前频率 */
    uint32_t rate = hal_clk_get_rate(clk);
    printf("Clock rate: %u Hz\n", rate);
} else {
    printf("Clock is disabled\n");
}

hal_clock_put(clk);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. **读取寄存器确认**

```
# 读取UART0时钟寄存器（以R128为例）
p 0x0200190C

# 输出示例：0x80000000
# Bit[31]=1   时钟已开门
# Bit[25:24]=00   时钟源选择APB1
# Bit[4:0]=0   分频系数为1
```

完整检查流程：

```
void check_uart_clock(void)
{
    hal_clk_t clk, parent;
    uint32_t rate;

    /* 1. 获取UART时钟 */
    clk = hal_clock_get(HAL_SUNXI_CCU, CLK_BUS_UART0);
    if (!clk) {
        printf("[FAIL] Get clock failed\n");
        return;
    }

    /* 2. 检查使能状态 */
    if (hal_clock_is_enabled(clk) == HAL_CLK_STATUS_ENABLED) {
        printf("[OK] Clock is enabled\n");
    } else {
        printf("[FAIL] Clock is disabled\n");
        goto cleanup;
    }

    /* 3. 检查时钟频率 */
    rate = hal_clk_get_rate(clk);
    printf("[INFO] Clock rate: %u Hz\n", rate);
    if (rate == 0) {
        printf("[FAIL] Clock rate is 0\n");
        goto cleanup;
    }

    /* 4. 检查父时钟 */
    parent = hal_clk_get_parent(clk);
    if (parent) {
        if (hal_clock_is_enabled(parent) == HAL_CLK_STATUS_ENABLED) {
            uint32_t parent_rate = hal_clk_get_rate(parent);
            printf("[OK] Parent clock enabled, rate: %u Hz\n", parent_rate);
        } else {
            printf("[FAIL] Parent clock is disabled\n");
        }
        hal_clock_put(parent);
    }

cleanup:
    hal_clock_put(clk);
}
```

6.2 一号通 FAQ 列表

更多问题请访问全志科技一号通平台，搜索 “CCU” 或 “时钟” 相关关键词。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2026 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。